CODEX 48 · ROUND 2 · THE BUILD

BrandGuard AI

Intelligent brand governance, before anything goes public

Codex 48 · Round 2 — The Build

Team	CHOOSE AS U WISH CYBER SQUAD
Team leader	Dhanraj Sonar
Live product	Pending deployment — add public URL
Source code	Pending push — add public repository URL
Updated PDF	`BrandGuard-AI-Round-2-Submission.pdf`

> **The product in one sentence:** a marketer pastes a draft, four specialist agents check it against the company's approved guidelines, claims, legal terms and visual assets, every issue comes back with an exact clause citation and a one-click fix, and a human decides whether to approve, override or publish.

No login, no setup, no API keys. The public demo opens directly into a working workspace with a non-compliant sample draft already loaded, so a judge can run the entire central path in about ninety seconds.

1. What we built

BrandGuard AI is a pre-publication governance workspace for marketing content. It is not a dashboard with a scan button bolted on; the review loop is the product:

1. **Content goes in** — paste a draft, load the sample, or upload an existing scan.
2. **Four agents inspect it** — Tone & Vocabulary, Fact & Claims, Visual Compliance and Accessibility run over the same draft in a single in-process pass.
3. **Issues come back explained** — each finding contains the offending text, character offsets for inline highlighting, severity, a plain-language explanation, the exact guideline clause (`code` , section and quoted text), and a context-preserving suggested fix.
4. **A human decides** — accept the fix and the draft is rewritten and re-checked; override with a written justification and the exception is remembered; a Manager, Admin or Legal role approves; only an approved draft can be published. Every action is written to an audit trail.

The demo ships with a realistic corpus: **16 guideline clauses**, five historical scans, seeded analytics, and a deliberately bad Q4 launch draft containing an unverified `500%` performance claim, a prohibited guarantee, hype vocabulary, an off-palette colour, an unsplash image and ambiguous link text. A fresh inspection returns **16 findings** across the four agents.

---

2. Try it in ninety seconds

Open the live link in a private window (no account needed):

1. The workspace loads the sample draft. Click **Run inspection**.
2. Sixteen findings appear. Open **Fact & Claims** — the critical claim cites *Approved Fact Sheet v2.1 §4.2* and points to the retired 500% figure in §4.5.
3. Click **Accept suggestion** on the claim. The draft becomes "up to 150%", the finding is marked Accepted, the score rises (44 → 54 in the seeded example) and the other findings are re-checked.
4. Click **Override** on another finding, enter a note such as "Approved for this campaign by Legal" and submit. Re-scan: the exception is suppressed and the decision is in **Activity**.
5. Switch the header role to **Manager**, then **Approve draft** → **Publish**.
6. Open **Insights** for live KPIs, the 30-day violation trend, inspector load, guideline clarity and the knowledge-base registry.
7. Press **Ctrl/Cmd+K** and search `logo` — the palette ranks `VIS-1.3` and opens it.

Guard rails are enforced by the API, not just hidden in the UI: a Creator gets `403` when approving, and unresolved critical findings return `409` until they are resolved.

---

3. List of completed features

3.1 The central path (scored workflow)

Capability	What works today	Where
Draft ingestion	Paste/edit in the workspace, channel / audience / region / campaign metadata, sample draft loader, 40k-character limit	`frontend/components/Workspace.tsx` , `POST /api/scans`

Capability	What works today	Where
Guideline-grounded inspection	Four agents check against 16 seeded clauses covering tone, lexicon, facts, legal, visual identity and WCAG	`backend/app/inspectors.py`
Exact citations	Each finding links to a guideline row and stores the clause code, section, and source quote	`backend/app/pipeline.py`
Explainable findings	Severity, plain-language explanation, offending segment, character offsets	findings API + UI cards
One-click fixes	Suggested rewrites preserve surrounding Markdown; accepting rewrites the draft and triggers a re-check	`/accept`
Human override	Mandatory written justification; overridden findings are suppressed on re-scan instead of reappearing	`/override`
Inline review	Split-pane canvas highlights the exact spans; clicking a highlight selects the finding	`frontend/components/DocumentCanvas.tsx`
Comments	Threaded review comments attached to a scan or a specific finding	`/comments`
Approval gate	Manager / Admin / Legal only; unresolved critical findings block approval	`/approve`
Publish gate	Requires an approved scan; role-checked; writes audit	`/publish`
Audit trail	Actor, role, action, detail and timestamp for create, accept, override, comment, approve, publish	`audit_events`, Activity tab
Re-scan	Re-runs all agents while preserving resolved and overridden decisions	`/api/scans/{id}` + UI

3.2 Guideline and knowledge management

- Guideline library with codes, sections, categories and tags (`GET/POST/DELETE /api/guidelines`).
- Policy upload (Markdown/TXT) with heading-aware chunking and immediate indexing (`POST /api/guidelines/upload`).
- TF-IDF retrieval over clauses with ranked, explainable matches (`GET /api/knowledge-base/search`).
- Global search palette (`Ctrl/Cmd+K`) for guideline clauses, with keyboard navigation.
- Knowledge-base registry in Insights showing chunks, overrides and a clarity score per clause.

3.3 Governance analytics

- Total scans, open findings, compliance rate, top violation and share, average resolution time.
- 30-day stacked violation trend by severity.
- Inspector load (findings per agent) and severity breakdown.
- Guideline clarity index (`100 − override rate`), sorted to expose ambiguous rules.
- Recent scans, live refresh, and deep links from every item back into the workspace.

3.4 Experience and platform

- Responsive dark workspace designed for a two-pane review, not a marketing page.
- Accessible semantics: landmarks, labels, focus states, skip link, contrast-safe palette.
- Deep link `/?scan=<id>` opens a specific review directly.
- Fully typed API client and TanStack Query caching/invalidation.

- OpenAPI docs at `/docs`.
- 11 end-to-end API tests covering inspection, citations, accepts, overrides, role gates, approval/publish and analytics.
- One-command local launcher (`scripts/dev.ps1`) and a single-container public deployment.

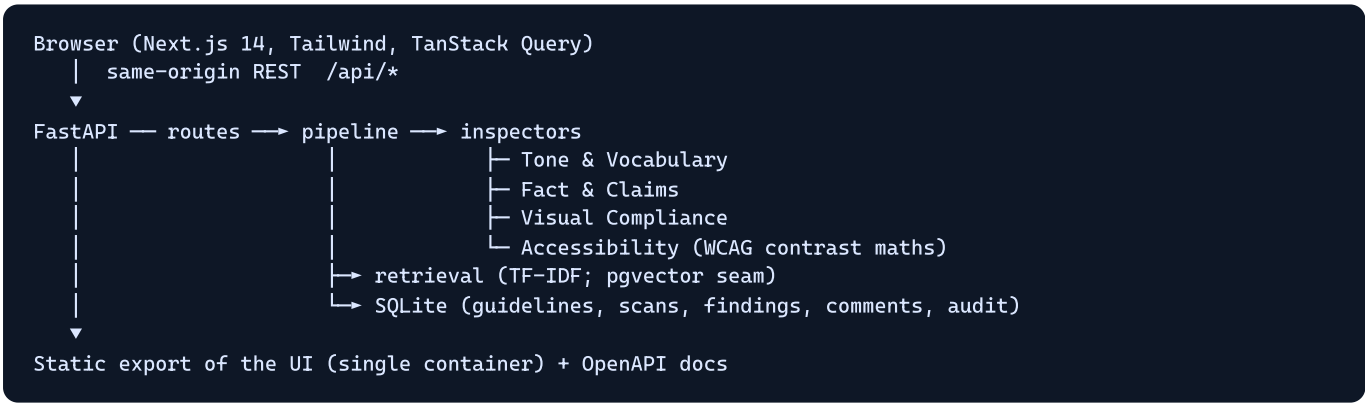
---

4. Technology and implementation details

4.1 Stack

Layer	Choice	Why
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS	Fast static export for one-URL deployment; typed UI; no runtime server needed
Data fetching	TanStack Query v5	Cache invalidation after every mutation; explicit loading/error states
Backend	Python 3.11+, FastAPI, Pydantic v2	Typed request models, OpenAPI for free, async-friendly
Inspection	Deterministic multi-agent rules	Reproducible citations, no keys, sub-second scans, testable
Retrieval	TF-IDF cosine over clause text	Offline relevance ranking behind a single seam ready for pgvector
Storage	SQLite (stdlib)	Zero infrastructure; schema mirrors the planned PostgreSQL model
Packaging	Multi-stage Docker, Next static export served by FastAPI	One public URL, same origin, no CORS failure mode

4.2 Architecture



4.3 Inspection engine

All agents implement one contract: `{agent, code, guideline_id, severity, issue_type, segment, start_index, end_index, explanation, suggested_fix}`. The pipeline joins findings to real guideline rows, so a citation cannot be invented by the agent.

Agent	Checks in the demo
Tone & Vocabulary	Hype/superlative vocabulary, unsupported leadership claims, non-approved lexicon, Flesch–Kincaid reading grade
Fact & Claims	Unverified performance metrics against the approved fact sheet, prohibited guarantees, outdated customer metrics, promotions missing disclaimers, stale dated claims

Agent	Checks in the demo
Visual Compliance	Logo clear-space risk, unauthorised imagery sources, off-palette colour values
Accessibility	Missing/insufficient alt text, WCAG contrast ratio computed from foreground/background, ambiguous link text

Severity is weighted (`Critical 10` , `Warning 3` , `Info 1`); the compliance score is `100 − unresolved penalties` , floored at zero. Overridden findings are removed from the penalty and act as suppressions on subsequent scans. Stale finding actions are rejected when the draft has changed underneath them (`409`), preventing "apply a fix to text that no longer exists".

The rule agents are deliberately deterministic for the hackathon: every finding is reproducible, every citation is real, and the demo runs with no network. The structured contract is the seam for Round 3's LLM agents — swapping the implementation does not touch the pipeline, API or UI.

4.4 Data model

Five tables mirror the planned PostgreSQL design: `guidelines` , `scans` , `findings` , `comments` , `audit_events` , with indexes on scan/date/audit lookups and cascading deletes. Every query is plain SQL; porting to Postgres + pgvector means replacing `db.py` and `retrieval.py` , not rewriting the product.

Notable fields: findings keep `start_index` / `end_index` for inline highlights and `guideline_id` for the citation join; scans keep channel/audience/region/campaign context used by the agents; audit rows keep actor and role for every mutation.

4.5 API surface

```
POST /api/scans                                create + inspect a draft
GET  /api/scans                                list reviews
GET  /api/scans/{id}                           scan, findings, citations, comments, audit
POST /api/scans/{id}/findings/{fid}/accept     apply fix + re-check
POST /api/scans/{id}/findings/{fid}/override   exception with justification
POST /api/scans/{id}/comments                  review comment
POST /api/scans/{id}/approve                   role-gated approval
POST /api/scans/{id}/publish                   role-gated publish
GET  /api/guidelines                           clause library
POST /api/guidelines/upload                    heading-aware policy ingestion
GET  /api/knowledge-base/search?q=             ranked retrieval
GET  /api/analytics                            KPIs, trend, clarity, KB stats
GET  /api/health                               liveness
```

4.6 One-URL deployment

`Dockerfile` builds the Next.js static export with `NEXT_STATIC_EXPORT=1` and `NEXT_PUBLIC_API_URL=/` , copies it into the FastAPI image, and FastAPI serves `/_next/*` , the two routes, `index.html` fallback for client-side paths, and `/api/*` from the same origin. The result is one container and one link, which removes the most common hackathon failure: a frontend deployed on one host calling `localhost` or a cold backend on another.

`render.yaml` deploys that image to Render's free tier with health checks. Full steps, including a private-window verification checklist, are in `DEPLOY.md` .

4.7 Quality evidence

- `pytest -q` — **11 passed**: inspection, citation integrity, accept/re-check, override suppression, Creator `403` , critical `409` , approval/publish, analytics, search, upload.
- `npm run build` — type-checks and produces the production bundle; static export builds all routes.
- Browser-verified flows (private session): run inspection → 16 findings, accept → score 44 → 54, override → suppressed, Creator blocked, Manager approve, publish, `Ctrl+K` search ranking, analytics KPIs, deep links, direct `/analytics`

refresh.

- Single container verified locally: `/`, `/analytics`, `/api/health`, assets and same-origin API calls all work with no `localhost` requests.

4.8 Privacy and security posture

- No customer content or secrets leave the machine/container; there are no model API calls.
- Demo data is synthetic ("Acme Corp"). The database is gitignored and reseeded on deploy.
- Role gates and state transitions are enforced server-side (`403` / `409`), not only in the UI.
- Input limits (40k characters), parameterised SQL, path-traversal checks on the static server.
- The demo intentionally has no authentication so judges can test every role; production identity is a Round 3 item below.

---

5. Changes from the original plan

The central promise did not change: **content in → guideline-grounded findings with citations → human decision**. The changes are about making that path work end-to-end, offline, in front of a judge.

Area	Original plan	Round 2 build	Why
Storage	PostgreSQL 16 + pgvector	SQLite with a schema mirroring the same tables	Zero infrastructure means the public demo and a fresh clone start with one command; the SQL and retrieval seams are preserved
Retrieval	Embedding vectors + pgvector cosine	TF-IDF cosine over clause text	No API keys, deterministic ranking, instant startup; same single retrieval interface to swap in Round 3
Inspection	LLM chains per agent	Deterministic rule agents behind the same structured finding contract	Reproducible citations, sub-second scans, no hallucinated clauses, no key leakage; LLM/VLM upgrade stays on the roadmap
Identity	OIDC/SSO and tenant RBAC	In-app role switcher, server-enforced role gates and audit actor/role	A judge must test Manager-only approval without creating an account; the authorization logic is real even though the identity provider is not
Deployment	Vercel frontend + separate API, Redis/Celery	One Docker container: Next static export served by FastAPI	One public link, no CORS, no split cold start, and no chance of a frontend pointing at `localhost`
Ingestion	PDF/DOCX/HTML via Unstructured + OCR	Markdown/TXT with heading-aware chunking	Keeps the demo dependency-free; the parser boundary is isolated for Round 3
Visual checks	Pixel/VLM analysis of uploaded assets	Markdown image references, palette values and contrast values	No binary asset pipeline in the timebox; the findings and UI already carry the right regions/severities for a real vision pipeline
Async work	Celery + Redis + WebSocket progress	Synchronous in-process scanning with a 40k-cap	Scans complete in well under a second; a queue adds failure modes without helping the judged path
Extras added	—	Inline highlight canvas, global search, audit/activity trail, seeded historical analytics, KB manager, deep links, 11 API tests	These make the central path inspectable and demonstrable, and they are real product surfaces rather than mock data

What this means for judging: no part of the core loop is faked. The sample is real content, the clauses are real rows, the citations come from those rows, accepting a fix really rewrites the draft, and overrides really change later scans. The deferred pieces are production hardening and scale, not the workflow.

Known limitations, stated plainly: no authentication provider, SQLite is ephemeral on the free tier, policy upload is text-only, visual inspection reasons over image references rather than pixels, and scans are synchronous. Each maps to a Round 3 improvement.

---

6. Three major improvements planned for Round 3

These are product additions, not redesigns. Each has a shippable scope and an acceptance test.

Improvement 1 — Production RAG and LLM inspection with real document ingestion

What: replace the demo's TF-IDF + rules with a grounded retrieval-augmented pipeline, and accept the formats brand teams actually use.

- Ingest PDF, DOCX, PPTX, HTML and scanned pages through Unstructured + OCR; preserve headings, tables and page numbers in chunk metadata.
- Store embeddings in PostgreSQL + pgvector; hybrid retrieval (dense + BM25) with clause versioning and duplicate suppression.
- Run each agent as a structured-output LLM call (JSON schema, temperature 0) with retrieved clauses as the only evidence. A quote-verification guard rejects any citation whose quoted text is not literally present in the retrieved clause.
- Keep the deterministic rule agents as a fallback and cross-check; low-confidence findings are routed to “needs human review” instead of being asserted.
- Build an evaluation harness (labelled drafts, precision/recall, citation validity, hallucination rate) and run it in CI before deploying prompt or model changes.

Why it matters: this is the difference between a demo corpus and a customer's real policy library. It also makes the product multilingual and robust to documents that are not clean Markdown.

Acceptance: PDF/DOCX parses retain page/section metadata; on a 200-draft labelled set, finding precision ≥ 0.90 and citation validity = 100%; p95 scan ≤ 8 s for 10k characters; every finding still resolves to a real clause; the rules fallback keeps the product usable if the model provider is down.

Improvement 2 — Visual brand-asset inspection

What: inspect the actual pixels of logos, campaign images and video frames against the brand system, not just Markdown image references.

- Upload assets or connect a DAM; generate thumbnails and video keyframes.
- Deterministic checks on pixels: logo bounding box and clear space relative to logo height, palette distance (ΔE2000) against approved tokens, rendered text/background contrast, OCR for text baked into images, and channel-specific safe-area/crop validation.
- A vision-language model adds semantic checks: unauthorised product depiction, competitor marks, inappropriate scenes, and suggested alt text.
- Findings carry bounding boxes/regions, so the review canvas can highlight the exact area of the asset, with accept/override flowing through the existing workflow.

Why it matters: brand violations are frequently visual, and a text-only inspection misses the most expensive mistakes (stretched logo, wrong palette, unreadable sale banner).

Acceptance: a 20-asset corpus produces measured clear-space, palette and contrast findings with regions; per-channel crop previews render; generated alt text meets WCAG guidance; visual findings use the same citation/approval/audit path as text findings.

Improvement 3 — Enterprise workflow, integrations and tenant isolation

What: make the review loop deployable inside a real organisation.

- OIDC/SSO (Google Workspace, Microsoft Entra) + SCIM provisioning; signed sessions.
- PostgreSQL row-level security per tenant, encrypted object storage, retention policies.
- Versioned policy packs with diff and a publish/approve flow for the guidelines themselves.
- Configurable approval chains (e.g. Legal required for pricing claims), SLA timers and escalation.
- Slack/Teams notifications with approve/override actions; CMS connectors (WordPress, Contentful, HubSpot) that block publication until the scan is approved; signed webhooks for other systems.
- Immutable, exportable audit log (CSV/JSON) with hash verification, plus per-team analytics.

Why it matters: governance only works if it sits in the tools where content is published and survives an audit. This turns the prototype into something a brand team can adopt.

Acceptance: two tenants cannot read each other's rows (automated isolation test); SSO login and role mapping work; a CMS draft cannot be published before approval; an approval chain routes and escalates correctly; the audit export verifies against its hash; Slack actions update status and write audit rows.

---

7. Repository and local run

```
backend/
  app/main.py        FastAPI routes, analytics, static UI serving
  app/pipeline.py    scan lifecycle, resolution, approval gates, audit
  app/inspectors.py  four agents + WCAG contrast maths
  app/retrieval.py   TF-IDF retrieval (pgvector seam)
  app/seed.py        16 clauses + historical demo scans
  app/db.py          SQLite schema mirroring the TRD tables
  tests/test_api.py  11 end-to-end API tests
frontend/
  app/               workspace and insights routes
  components/        split-pane canvas, findings, header/search, analytics
  lib/               typed API client, types, sample draft
Dockerfile           Next export + FastAPI single image
render.yaml          free-tier Render blueprint
docs/                deployment guide + this submission
scripts/dev.ps1      one-command local launcher
```

```
# local, two processes
.\scripts\dev.ps1                 # API :8000, UI :3000

# or the exact public shape: one container
docker build -t brandguard-ai .
docker run --rm -p 8000:8000 brandguard-ai
# open http://localhost:8000
```

Reset the demo by deleting `backend/data/brandguard.db` and restarting the API. The public container reseeds on every deploy/restart.

---

8. Submission checklist (team leader)

- [ ] Live link opens in a **private window** with no sign-in and no walkthrough.
- [ ] Source repository is **public** and opens in a private window.
- [ ] This PDF is uploaded and the two links above are the final deployed URLs.
- [ ] Completed-feature list, technology details, plan changes and three Round 3 improvements are included (sections 3–6).
- [ ] Only the team leader files the submission; members do not submit separately.
- [ ] Submitted before **Wednesday 28 October, 10:00 AM IST** (hard deadline).
- [ ] Live URL warmed up (open it a few minutes early) before any demo or judging window.

BrandGuard AI — built for Codex 48 Round 2. Content in, evidence out, human decides.